

EJERCICIO PRÁCTICO: "Aprendiendo Prompts "

PARTE 1 LA BASE

Un prompt es como dar instrucciones a un ayudante

1. Entre más específico, mejor el resultado
2. Se puede pedir: código, explicaciones, refactorizar, documentar

```
# Archivo: calculadora_basica.py
# ESTO ES LO QUE TÚ LES MUESTRAS COMPLETO

def suma(a, b):
    """Suma dos números"""
    return a + b

def resta(a, b):
    """Resta dos números"""
    return a - b

# PUNTO DE PARTIDA PARA LOS ALUMNOS:
# A partir de aquí, ellos usarán prompts para continuar

def multiplicacion(a, b):
    # TODO: Los alumnos pedirán a Copilot que complete esto
    pass

def division(a, b):
    # TODO: Los alumnos pedirán a Copilot que complete esto
    pass
```

Respuesta:

Prompt que utilizaria para la multiplicacion seria algo asi, completame la funcion multiplicacion (a,b) para que multiplique ambos numero y devuelva el resultado y quedaria algo como esto:

```
</> Python

def multiplicacion(a, b):
    """Multiplica dos números"""
    return a * b
```

El prompt para la division seria casi igual solo añadiremos que no se pueda dividir entre 0 ej: Completame la funcion division (A,B) para que divida ambos numero y devuelva el resultado. Pero aqui añadiremos si B es 0 lanza un ValueError con el mensaje "no se puede dividir por cero" y quedaria algo como esto:

```
</> Python

def division(a, b):
    """Divide dos números"""
    if b == 0:
        raise ValueError("No se puede dividir por cero")
    return a / b
```

PARTE 2: EJERCICIOS PRÁCTICOS

HOJA DE TRABAJO

EJERCICIO 1: Generar una función con prompt

Instrucción: Copia este prompt exactamente:

```
"Escribe una función en Python que valide emails. La función debe:
1. Verificar que tenga un @
2. Verificar que tenga un punto después del @
3. Verificar que no tenga espacios
4. Retornar True si es válido, False si no
5. Incluir ejemplos de uso"
```

Resultado esperado que generará:

```
def validar_email(email):
    """
    Valida si un email tiene formato correcto

    Args:
        email (str): Email a validar

    Returns:
        bool: True si es válido, False si no
    """
    # Verificar que tenga @
    if '@' not in email:
        return False

    # Separar usuario y dominio
    partes = email.split('@')
    if len(partes) != 2:
        return False

    usuario, dominio = partes
```

```
# Verificar que no haya espacios
if ' ' in email:
    return False

# Verificar que haya un punto en el dominio
if '.' not in dominio:
    return False

# Verificar que el dominio tenga al menos 2 caracteres después del punto
if len(dominio.split('.')[1]) < 2:
    return False

return True

# Ejemplos de uso
if __name__ == "__main__":
    print(validar_email("usuario@example.com")) # True
    print(validar_email("usuario@example"))    # False
    print(validar_email("usuario example.com")) # False
```

Respuesta: Esto es lo que me ha dado

```
Python

def validar_email(email):
    """
    Valida si un email tiene formato básico correcto.

    Reglas:
    - Debe contener un '@'
    - Debe contener un punto después del '@'
    - No debe contener espacios
    - Retorna True si es válido, False si no
    """

    # Verificar que no haya espacios
    if " " in email:
        return False

    # Verificar que tenga '@'
    if "@" not in email:
        return False

    # Separar usuario y dominio
    partes = email.split("@")
    if len(partes) != 2:
        return False

    usuario, dominio = partes

    # Verificar que haya un punto en el dominio
    if "." not in dominio:
        return False

    return True

# Ejemplos de uso
if __name__ == "__main__":
    print(validar_email("usuario@ejemplo.com")) # True
    print(validar_email("usuari@ejemplo.com")) # False
    print(validar_email("usuario@ejemplo")) # False
    print(validar_email("usuario ejemplo@ejemplo.com")) # False
```

EJERCICIO 2: Refactorizar código

Instrucción: Copia este código y este prompt:

```
"Refactoriza esta función para mejorar rendimiento y legibilidad.  
Explica los cambios que hiciste y por qué son mejores."
```

```
def procesar_datos(lista):  
    """  
    Procesa una lista de números: duplica los pares, triplica los impares.  
  
    Refactorizaciones realizadas:  
    1. Uso de list comprehension en lugar de loop tradicional → Más legible y Pythonico  
    2. Uso de operador ternario → Más conciso  
    3. Nombre más descriptivo en la variable temporal  
    4. Eliminación del índice innecesario  
  
    Args:  
        lista (list): Lista de números a procesar  
  
    Returns:  
        list: Lista procesada  
    """  
    return [num * 2 if num % 2 == 0 else num * 3 for num in lista]  
  
# Versión aún más eficiente si necesitamos procesar muchos datos:  
def procesar_datos_v2(lista):  
    """Versión con map para mejor rendimiento con listas grandes"""  
    return list(map(lambda x: x * 2 if x % 2 == 0 else x * 3, lista))
```

Cambios:

```
Python  
def procesar_datos(numeros):  
    """  
    Procesa una lista de números:  
    - Duplica los números pares  
    - Triplica los números impares  
  
    Args:  
        numeros (list[int]): Lista de números a procesar  
  
    Returns:  
        list[int]: Lista procesada  
    """  
    return [n * 2 if n % 2 == 0 else n * 3 for n in numeros]
```

Explicación de que he hecho con la ayuda de la IA.

1)Nombre de variable más claro

- Antes: lista.
- Ahora: numeros.

- ✓Más descriptivo.
- ✓Mejora legibilidad.

2)Uso de list comprehension

En lugar de usar un for tradicional con append, se usa.

```
Python  
[n * 2 if n % 2 == 0 else n * 3 for n in numeros]
```

- ✓Más compacto.
- ✓Más Pythonico.
- ✓Más rápido que crear lista manualmente con append.

3)Eliminación de código innecesario

Se eliminan:

- Variables temporales.
- Índices innecesarios.
- Comentarios redundantes.

- ✓Código más limpio.
- ✓Más fácil de mantener.

Versión alternativa para grandes volúmenes de datos

Si realmente se procesan MUCHOS datos:

```
Python

def procesar_datos_iterable(numeros):
    """Versión con generador para grandes volúmenes de datos"""
    for n in numeros:
        yield n * 2 if n % 2 == 0 else n * 3
```

✓No crea lista completa en memoria.

✓Mejor rendimiento para datasets grandes.

EJERCICIO 3: Documentar código

Instrucción: Copia este código y prompt:

```
"Genera documentación estilo JSDoc/Python Docstring para esta clase.
Incluye descripción de la clase, cada método, parámetros y valores de retorno."
```

Resultado esperado:

```
class Usuario:
    """
    Clase que representa a un usuario del sistema.

    Atributos:
        nombre (str): Nombre completo del usuario
        edad (int): Edad del usuario en años
        email (str): Dirección de correo electrónico
        activo (bool): Estado de activación del usuario
    """

    def __init__(self, nombre, edad, email):
        """
        Inicializa una nueva instancia de Usuario.

        Args:
            nombre (str): Nombre completo del usuario
            edad (int): Edad del usuario en años
            email (str): Dirección de correo electrónico válida

        Returns:
```

```
    Returns:
        Usuario: Instancia de usuario creada
    """

    self.nombre = nombre
    self.edad = edad
    self.email = email
    self.activo = True

def cambiar_email(self, nuevo_email):
    """
    Cambia el email del usuario si es válido.

    Args:
        nuevo_email (str): Nueva dirección de email

    Returns:
        bool: True si el cambio fue exitoso, False si el email no es válido
    """
    if "@" in nuevo_email:
```

```
        if "@" in nuevo_email:
            self.email = nuevo_email
            return True
        return False

def desactivar(self):
    """
    Desactiva la cuenta del usuario.

    Returns:
        bool: Siempre retorna True indicando que la operación fue exitosa
    """
    self.activo = False
    return True
```

Respuesta: Estilo Python Docstring.

```
Python
class Usuario:
    """
    Representa a un usuario del sistema.

    Attributes:
        nombre (str): Nombre completo del usuario.
        edad (int): Edad del usuario en años.
        email (str): Dirección de correo electrónico del usuario.
        activo (bool): Estado de activación del usuario.
    """

    def __init__(self, nombre: str, edad: int, email: str):
        """
        Inicializa una nueva instancia de Usuario.

        Args:
            nombre (str): Nombre completo del usuario.
            edad (int): Edad del usuario en años.
            email (str): Dirección de correo electrónico válida.

        Returns:
            None
        """
        self.nombre = nombre
        self.edad = edad
        self.email = email
        self.activo = True

    def cambiar_email(self, nuevo_email: str) -> bool:
        """
        Cambia el email del usuario si es válido.

        Args:
            nuevo_email (str): Nueva dirección de correo electrónico.

        Returns:
            bool: True si el cambio fue exitoso,
                  False si el email no es válido.
        """
        if "@" in nuevo_email:
            self.email = nuevo_email
            return True
        return False

    def desactivar(self) -> bool:
        """
        Desactiva la cuenta del usuario.

        Returns:
            bool: Siempre retorna True indicando que
                  la operación fue exitosa.
        """
        self.activo = False
        return True
```

Qué incluye esta documentación

- ✓ Descripción clara de la clase.
- ✓ Documentación de atributos.
- ✓ Documentación del constructor.
- ✓ Documentación de cada método.
- ✓ Tipado de parámetros.
- ✓ Tipo de retorno explícito.

Estilo JSDoc: Este no me gusto para nada

```
Python

class Usuario:
    """
    @class Usuario
    @classdesc Representa a un usuario del sistema.

    @property {str} nombre - Nombre completo del usuario.
    @property {int} edad - Edad del usuario en años.
    @property {str} email - Dirección de correo electrónico del usuario.
    @property {bool} activo - Estado de activación del usuario.
    """

    def __init__(self, nombre, edad, email):
        """
        @constructor
        @param {str} nombre - Nombre completo del usuario.
        @param {int} edad - Edad del usuario en años.
        @param {str} email - Dirección de correo electrónico válida.
        @returns {None}
        """
        self.nombre = nombre
        self.edad = edad
        self.email = email
        self.activo = True

    def cambiar_email(self, nuevo_email):
        """
        Cambia el email del usuario si es válido.

        @param {str} nuevo_email - Nueva dirección de correo electrónico.
        @returns {bool} True si el cambio fue exitoso, False si el email no es válido.
        """
        if "@" in nuevo_email:
            self.email = nuevo_email
            return True
        return False

    def desactivar(self):
        """
        Desactiva la cuenta del usuario.

        @returns {bool} True indicando que la operación fue exitosa.
        """
        self.activo = False
        return True
```

Qué tiene estilo JSDoc

- ✓@class.✓@classdesc.
- ✓@property.✓@returns.
- ✓@constructor.✓@param.

PARTE 3: DESAFÍOS PARA CONTINUAR

DESAFÍO 1: Extender la calculadora

"Amplía la calculadora_basica.py para incluir:

1. Función potencia(base, exponente)
2. Función raiz_cuadrada(numero) con manejo de números negativos
3. Función calcular_porcentaje(porcentaje, total)
4. Un menú interactivo que permita elegir operación
5. Manejo de errores con try-except"

DESAFÍO 2: Crear un mini-juego

Prompt

"Crea un juego de 'Adivina el número' donde:

1. La computadora elige un número aleatorio entre 1-100
2. El usuario tiene 7 intentos
3. Da pistas 'más alto' o 'más bajo'
4. Lleva registro de puntuación
5. Pregunta si quiere jugar otra vez al terminar"

DESAFÍO 3: Sistema de inventario

Prompt:

bash

"Crea un sistema de inventario para una tienda con:

1. Clase Producto con nombre, precio, cantidad
2. Clase Inventario que maneje múltiples productos
3. Funciones para: agregar, eliminar, buscar, listar productos
4. Calcular valor total del inventario
5. Guardar y cargar de un archivo JSON"

DESAFIO1:

```
Python Obtener Plus x
import math

# ----- FUNCIONES ----- #

def suma(a, b):
    return a + b

def resta(a, b):
    return a - b

def multiplicacion(a, b):
    return a * b

def division(a, b):
    if b == 0:
        raise ValueError("No se puede dividir entre 0")
    return a / b

def potencia(base, exponente):
    return base ** exponente

def raiz_cuadrada(numero):
    if numero < 0:
        raise ValueError("No se puede calcular la raíz de un número negativo")
    return math.sqrt(numero)

def calcular_porcentaje(porcentaje, total):
    return (porcentaje / 100) * total

# ----- MENÚ ----- #

def menu():
    while True:
        print("\n=== CALCULADORA ===")
        print("1. Suma")
        print("2. Resta")
        print("3. Multiplicación")
        print("4. División")
        print("5. Potencia")
        print("6. Raíz Cuadrada")
        print("7. Calcular Porcentaje")
        print("8. Salir")

        opcion = input("Elige una opción: ")

        try:
            if opcion == "8":
                print("Hasta luego 🍀")
                break

            if opcion == "6":
                numero = float(input("Número: "))
                print("Resultado:", raiz_cuadrada(numero))
```

```

elif opcion == "7":
    porcentaje = float(input("Porcentaje: "))
    total = float(input("Total: "))
    print("Resultado:", calcular_porcentaje(porcentaje, total))
else:
    a = float(input("Primer número: "))
    b = float(input("Segundo número: "))

    operaciones = {
        "1": suma,
        "2": resta,
        "3": multiplicacion,
        "4": division,
        "5": potencia
    }

    if opcion in operaciones:
        print("Resultado:", operaciones[opcion](a, b))
    else:
        print("Opción inválida")

except ValueError as e:
    print("Error:", e)
except Exception:
    print("Ocurrió un error inesperado")

```

menu()

DESAFIO 2

```
Python

import random

def jugar():
    numero_secreto = random.randint(1, 100)
    intentos = 7
    puntuacion = 0

    print("\n🎲 Adivina el número entre 1 y 100")

    while intentos > 0:
        try:
            intento = int(input(f"Te quedan {intentos} intentos: "))

            if intento == numero_secreto:
                print("🎉 ¡Correcto!")
                puntuacion = intentos * 10
                break
            elif intento < numero_secreto:
                print("📉 Más alto")
            else:
                print("📈 Más bajo")

            intentos -= 1

        except ValueError:
            print("Introduce un número válido")

    if intentos == 0:
        print("❌ Perdiste. El número era:", numero_secreto)

    print("🏆 Puntuación:", puntuacion)

while True:
    jugar()
    repetir = input("¿Quieres jugar otra vez? (s/n): ").lower()
    if repetir != "s":
        print("Gracias por jugar 🙌")
        break
```

DESAFIO 3

```
Python

import json

class Producto:
    def __init__(self, nombre, precio, cantidad):
        self.nombre = nombre
        self.precio = precio
        self.cantidad = cantidad

    def to_dict(self):
        return {
            "nombre": self.nombre,
            "precio": self.precio,
            "cantidad": self.cantidad
        }

class Inventario:
    def __init__(self):
        self.productos = []

    def agregar_producto(self, producto):
        self.productos.append(producto)

    def eliminar_producto(self, nombre):
        self.productos = [p for p in self.productos if p.nombre != nombre]

    def buscar_producto(self, nombre):
        for p in self.productos:
            if p.nombre == nombre:
                return p
        return None

    def listar_productos(self):
        for p in self.productos:
            print(f"{p.nombre} - Precio: {p.precio} - Cantidad: {p.cantidad}")

    def valor_total(self):
        return sum(p.precio * p.cantidad for p in self.productos)

    def guardar(self, archivo):
        with open(archivo, "w") as f:
            json.dump([p.to_dict() for p in self.productos], f, indent=4)

    def cargar(self, archivo):
        try:
            with open(archivo, "r") as f:
                datos = json.load(f)
                self.productos = [
                    Producto(d["nombre"], d["precio"], d["cantidad"])
                    for d in datos
                ]
```

```
    ]
    except FileNotFoundError:
        print("Archivo no encontrado, se creará uno nuevo.")

# ----- USO ----- #

inv = Inventario()
inv.cargar("inventario.json")

inv.agregar_producto(Producto("Teclado", 25, 10))
inv.agregar_producto(Producto("Mouse", 15, 20))

inv.listar_productos()
print("Valor total:", inv.valor_total())

inv.guardar("inventario.json")
```

ANGEL DAVID ECHENIQUE VILLANUEVA

Z0271067V

PROF: RUSGAR